

ML-1 Mini Project: Data Pre-processing and EDA

Topic: Autism Prediction

Name: Krisha Shah

Sap Id: 60009220055

Roll No: D065

```
In [ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [ ]: training_data = pd.read_csv('C:\\Krisha\\DJ Sem4\\ML\\Mini Project\\Autism Predicti
```

PRE-PROCESSING

```
In [ ]: training_data.head(10)
```

Out[]:

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	A7_Score	A8_Score
0	1	1	0	1	0	1	0	1	0
1	2	0	0	0	0	0	0	0	0
2	3	1	1	1	1	1	1	1	1
3	4	0	0	0	0	0	0	0	0
4	5	0	0	0	0	0	0	0	0
5	6	1	0	0	0	0	1	0	0
6	7	1	0	0	0	0	0	1	1
7	8	1	1	1	1	1	1	1	0
8	9	1	1	1	1	0	0	0	1
9	10	0	0	0	0	0	0	0	1

10 rows × 22 columns



In []:

```
training_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 22 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                     800 non-null    int64
1   A1_Score               800 non-null    int64
2   A2_Score               800 non-null    int64
3   A3_Score               800 non-null    int64
4   A4_Score               800 non-null    int64
5   A5_Score               800 non-null    int64
6   A6_Score               800 non-null    int64
7   A7_Score               800 non-null    int64
8   A8_Score               800 non-null    int64
9   A9_Score               800 non-null    int64
10  A10_Score              800 non-null    int64
11  age                    800 non-null    float64
12  gender                 800 non-null    object
13  ethnicity              800 non-null    object
14  jaundice               800 non-null    object
15  austim                 800 non-null    object
16  contry_of_res          800 non-null    object
17  used_app_before        800 non-null    object
18  result                 800 non-null    float64
19  age_desc               800 non-null    object
20  relation               800 non-null    object
21  Class/ASD              800 non-null    int64
dtypes: float64(2), int64(12), object(8)
memory usage: 137.6+ KB
```

```
In [ ]: training_data.describe()
```

Out[]:

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	A7_Score
count	800.0000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000
mean	400.5000	0.560000	0.530000	0.450000	0.41500	0.395000	0.303750	0.280000
std	231.0844	0.496697	0.499411	0.497805	0.49303	0.489157	0.460164	0.450000
min	1.0000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000
25%	200.7500	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000
50%	400.5000	1.000000	1.000000	0.000000	0.00000	0.000000	0.000000	0.000000
75%	600.2500	1.000000	1.000000	1.000000	1.00000	1.000000	1.000000	1.000000
max	800.0000	1.000000	1.000000	1.000000	1.00000	1.000000	1.000000	1.000000

```
In [ ]: training_data.shape
```

Out[]: (800, 22)

```
In [ ]: training_data.isnull().sum()
```

```
Out[ ]: ID          0
        A1_Score    0
        A2_Score    0
        A3_Score    0
        A4_Score    0
        A5_Score    0
        A6_Score    0
        A7_Score    0
        A8_Score    0
        A9_Score    0
        A10_Score   0
        age         0
        gender      0
        ethnicity   0
        jaundice    0
        austin      0
        contry_of_res 0
        used_app_before 0
        result      0
        age_desc    0
        relation    0
        Class/ASD   0
        dtype: int64
```

No null values in the above dataset

```
In [ ]: duplicate_rows = training_data[training_data.duplicated()]
        if duplicate_rows.empty:
            print("No duplicate rows found.")
        else:
            print("Duplicate Rows:")
            print(duplicate_rows)
```

No duplicate rows found.

```
In [ ]: categorical_columns = training_data.select_dtypes(include=['object']).columns
        inconsistent_values = {}
        for col in categorical_columns:
            unique_values = training_data[col].unique()
            if len(unique_values) > 1:
                inconsistent_values[col] = unique_values
        if inconsistent_values:
            print("Inconsistent Values:")
            for col, values in inconsistent_values.items():
                print(f"{col}: {values}")
        else:
            print("No inconsistent values found.")
```

```

Inconsistent Values:
gender: ['f' 'm']
ethnicity: ['?' 'White-European' 'Middle Eastern ' 'Pasifika' 'Black' 'Others'
            'Hispanic' 'Asian' 'Turkish' 'South Asian' 'Latino' 'others']
jaundice: ['no' 'yes']
austim: ['no' 'yes']
contry_of_res: ['Austria' 'India' 'United States' 'South Africa' 'Jordan'
                'United Kingdom' 'Brazil' 'New Zealand' 'Canada' 'Kazakhstan'
                'United Arab Emirates' 'Australia' 'Ukraine' 'Iraq' 'France' 'Malaysia'
                'Viet Nam' 'Egypt' 'Netherlands' 'Afghanistan' 'Oman' 'Italy'
                'AmericanSamoa' 'Bahamas' 'Saudi Arabia' 'Ireland' 'Aruba' 'Sri Lanka'
                'Russia' 'Bolivia' 'Azerbaijan' 'Armenia' 'Serbia' 'Ethiopia' 'Sweden'
                'Iceland' 'Hong Kong' 'Angola' 'China' 'Germany' 'Spain' 'Tonga'
                'Pakistan' 'Iran' 'Argentina' 'Japan' 'Mexico' 'Nicaragua' 'Sierra Leone'
                'Czech Republic' 'Niger' 'Romania' 'Cyprus' 'Belgium' 'Burundi'
                'Bangladesh']
used_app_before: ['no' 'yes']
relation: ['Self' 'Relative' 'Parent' '?' 'Others' 'Health care professional']

```

```

In [ ]: training_data['gender'] = training_data['gender'].map({'f': 'Female', 'm': 'Male'})
training_data['ethnicity'] = training_data['ethnicity'].replace('others', 'Others')
training_data['relation'] = training_data['relation'].replace('?', 'Unknown')

```

```

In [ ]: training_data.columns

```

```

Out[ ]: Index(['ID', 'A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
              'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age',
              'gender', 'ethnicity', 'jaundice', 'austim', 'contry_of_res',
              'used_app_before', 'result', 'age_desc', 'relation', 'Class/ASD'],
              dtype='object')

```

```

In [ ]: # Replace '?' with NaN
training_data.replace('?', np.nan, inplace=True)

# Drop rows with NaN values
training_data.dropna(inplace=True)

# Display the modified DataFrame
print(training_data)

```

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	\
2	3	1	1	1	1	1	1	
5	6	1	0	0	0	0	1	
6	7	1	0	0	0	0	0	
7	8	1	1	1	1	1	1	
8	9	1	1	1	1	0	0	
..	...	...	...	...	...	...	...	
790	791	1	0	1	1	1	1	
791	792	0	0	0	1	1	1	
795	796	0	1	0	0	0	0	
796	797	0	1	1	0	0	1	
797	798	0	0	0	0	0	0	

	A7_Score	A8_Score	A9_Score	...	gender	ethnicity	jaundice	\
2	1	1	1	...	Male	White-European	no	
5	0	0	1	...	Male	Middle Eastern	no	
6	1	1	1	...	Female	Pasifika	no	
7	1	0	1	...	Male	Black	no	
8	0	1	1	...	Male	White-European	no	
..	...	...	...	...	...	...	...	
790	1	0	1	...	Male	Others	no	
791	0	0	0	...	Male	Black	no	
795	0	0	1	...	Male	Hispanic	no	
796	0	1	1	...	Male	White-European	no	
797	0	0	0	...	Male	South Asian	yes	

	austim	contry_of_res	used_app_before	result	age_desc	relation	\
2	yes	United States		no 14.851484	18 and more	Self	
5	no	Jordan		no 9.562117	18 and more	Self	
6	no	United Kingdom		no 7.984569	18 and more	Self	
7	yes	United States		no 13.237898	18 and more	Self	
8	no	Brazil		no -1.755774	18 and more	Self	
..	...	...	...	...	...	...	
790	no	India		no 4.846236	18 and more	Self	
791	no	Australia		no 8.687281	18 and more	Self	
795	no	New Zealand		no 12.999501	18 and more	Self	
796	no	Cyprus		no 13.561518	18 and more	Self	
797	no	New Zealand		no 2.653177	18 and more	Self	

	Class/ASD
2	1
5	0
6	0
7	1
8	0
..	...
790	0
791	0
795	0
796	0
797	0

[597 rows x 22 columns]

Therefore the dataset is clean and can be worked upon.

Label Encoding to convert 'gender' values to numeric to use them in the EDA section

```
In [ ]: from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
training_data['gender_encoded'] = label_encoder.fit_transform(training_data['gender'])

print(training_data[['gender', 'gender_encoded']])
```

	gender	gender_encoded
2	Male	1
5	Male	1
6	Female	0
7	Male	1
8	Male	1
..	...	...
790	Male	1
791	Male	1
795	Male	1
796	Male	1
797	Male	1

[597 rows x 2 columns]

Hence we see that after encoding, all the 'Male' values have been encoded to 1 and 'Female' values to 0.

Label Encoding to convert country names to numeric data type.

```
In [ ]: label_encoder_country = LabelEncoder()
training_data['country_encoded'] = label_encoder_country.fit_transform(training_data['country_of_res'])

# Display the modified DataFrame
print(training_data[['country_of_res', 'country_encoded']])
```

	country_of_res	country_encoded
2	United States	53
5	Jordan	30
6	United Kingdom	52
7	United States	53
8	Brazil	12
..	...	...
790	India	24
791	Australia	5
795	New Zealand	35
796	Cyprus	16
797	New Zealand	35

[597 rows x 2 columns]

```
In [ ]: training_data['country_of_res'].unique()
```

```
Out[ ]: array(['United States', 'Jordan', 'United Kingdom', 'Brazil',
               'New Zealand', 'United Arab Emirates', 'India', 'Kazakhstan',
               'Iraq', 'France', 'Malaysia', 'Viet Nam', 'Egypt', 'Netherlands',
               'Canada', 'Australia', 'Afghanistan', 'Oman', 'Italy', 'Bahamas',
               'Saudi Arabia', 'Aruba', 'Russia', 'Bolivia', 'Azerbaijan',
               'Armenia', 'Serbia', 'Ethiopia', 'Sri Lanka', 'Sweden', 'Austria',
               'Iceland', 'Hong Kong', 'China', 'Germany', 'Spain', 'Tonga',
               'Pakistan', 'Iran', 'Argentina', 'South Africa', 'Japan', 'Mexico',
               'Ireland', 'Nicaragua', 'Sierra Leone', 'AmericanSamoa', 'Ukraine',
               'Czech Republic', 'Niger', 'Romania', 'Cyprus', 'Belgium',
               'Burundi', 'Bangladesh'], dtype=object)
```

```
In [ ]: # Assuming you have a DataFrame called training_data with columns 'ethnicity' and '
unique_countries = training_data['contry_of_res'].unique()
country_encoded = {}

# Assign encoded values to each unique ethnicity
for idx, contry_of_res in enumerate(unique_countries):
    country_encoded[contry_of_res] = idx

# Print unique encoded values for ethnicity
print("Unique encoded values for country:")
for contry_of_res, encoded_value in country_encoded.items():
    print(f"{contry_of_res}: {encoded_value}")
```


Unique encoded values for country:

United States: 0

Jordan: 1

United Kingdom: 2

Brazil: 3

New Zealand: 4

United Arab Emirates: 5

India: 6

Kazakhstan: 7

Iraq: 8

France: 9

Malaysia: 10

Viet Nam: 11

Egypt: 12

Netherlands: 13

Canada: 14

Australia: 15

Afghanistan: 16

Oman: 17

Italy: 18

Bahamas: 19

Saudi Arabia: 20

Aruba: 21

Russia: 22

Bolivia: 23

Azerbaijan: 24

Armenia: 25

Serbia: 26

Ethiopia: 27

Sri Lanka: 28

Sweden: 29

Austria: 30

Iceland: 31

Hong Kong: 32

China: 33

Germany: 34

Spain: 35

Tonga: 36

Pakistan: 37

Iran: 38

Argentina: 39

South Africa: 40

Japan: 41

Mexico: 42

Ireland: 43

Nicaragua: 44

Sierra Leone: 45

American Samoa: 46

Ukraine: 47

Czech Republic: 48

Niger: 49

Romania: 50

Cyprus: 51

Belgium: 52

Burundi: 53

Bangladesh: 54

```
In [ ]: label_encoder_ethnicity = LabelEncoder()
training_data['ethnicity_encoded'] = label_encoder_country.fit_transform(training_data['ethnicity'])

# Display the modified DataFrame
print(training_data[['ethnicity', 'ethnicity_encoded']])
```

	ethnicity	ethnicity_encoded
2	White-European	9
5	Middle Eastern	4
6	Pasifika	6
7	Black	1
8	White-European	9
..	...	...
790	Others	5
791	Black	1
795	Hispanic	2
796	White-European	9
797	South Asian	7

[597 rows x 2 columns]

```
In [ ]: training_data['ethnicity'].unique()
```

```
Out[ ]: array(['White-European', 'Middle Eastern', 'Pasifika', 'Black', 'Others',
              'Hispanic', 'Asian', 'Turkish', 'South Asian', 'Latino'],
              dtype=object)
```

```
In [ ]: # Assuming you have a DataFrame called training_data with columns 'ethnicity' and '
unique_ethnicities = training_data['ethnicity'].unique()
ethnicity_encoded = {}

# Assign encoded values to each unique ethnicity
for idx, ethnicity in enumerate(unique_ethnicities):
    ethnicity_encoded[ethnicity] = idx

# Print unique encoded values for ethnicity
print("Unique encoded values for ethnicity:")
for ethnicity, encoded_value in ethnicity_encoded.items():
    print(f"{ethnicity}: {encoded_value}")
```

Unique encoded values for ethnicity:

White-European: 0
Middle Eastern: 1
Pasifika: 2
Black: 3
Others: 4
Hispanic: 5
Asian: 6
Turkish: 7
South Asian: 8
Latino: 9

```
In [ ]: label_encoder_jaundice = LabelEncoder()
training_data['jaundice_encoded'] = label_encoder.fit_transform(training_data['jaundice'])

print(training_data[['jaundice', 'jaundice_encoded']])
```

```
label_encoder_autism = LabelEncoder()
training_data['austim_encoded'] = label_encoder.fit_transform(training_data['austim'])

print(training_data[['austim', 'austim_encoded']])
```

```
   jaundice  jaundice_encoded
2        no                0
5        no                0
6        no                0
7        no                0
8        no                0
..      ...                ...
790       no                0
791       no                0
795       no                0
796       no                0
797      yes                1
```

[597 rows x 2 columns]

```
   austim  austim_encoded
2      yes                1
5       no                0
6       no                0
7      yes                1
8       no                0
..      ...                ...
790     no                0
791     no                0
795     no                0
796     no                0
797     no                0
```

[597 rows x 2 columns]

```
In [ ]: output = []
        for col in training_data.columns:
            unique = training_data[col].nunique()
            colType = str(training_data[col].dtype)
            categories=training_data[col].unique()

            output.append([col, unique, colType,categories])

        output = pd.DataFrame(output)
        output.columns = ['colName','unique','dtype','categories']
        output
```

Out[]:

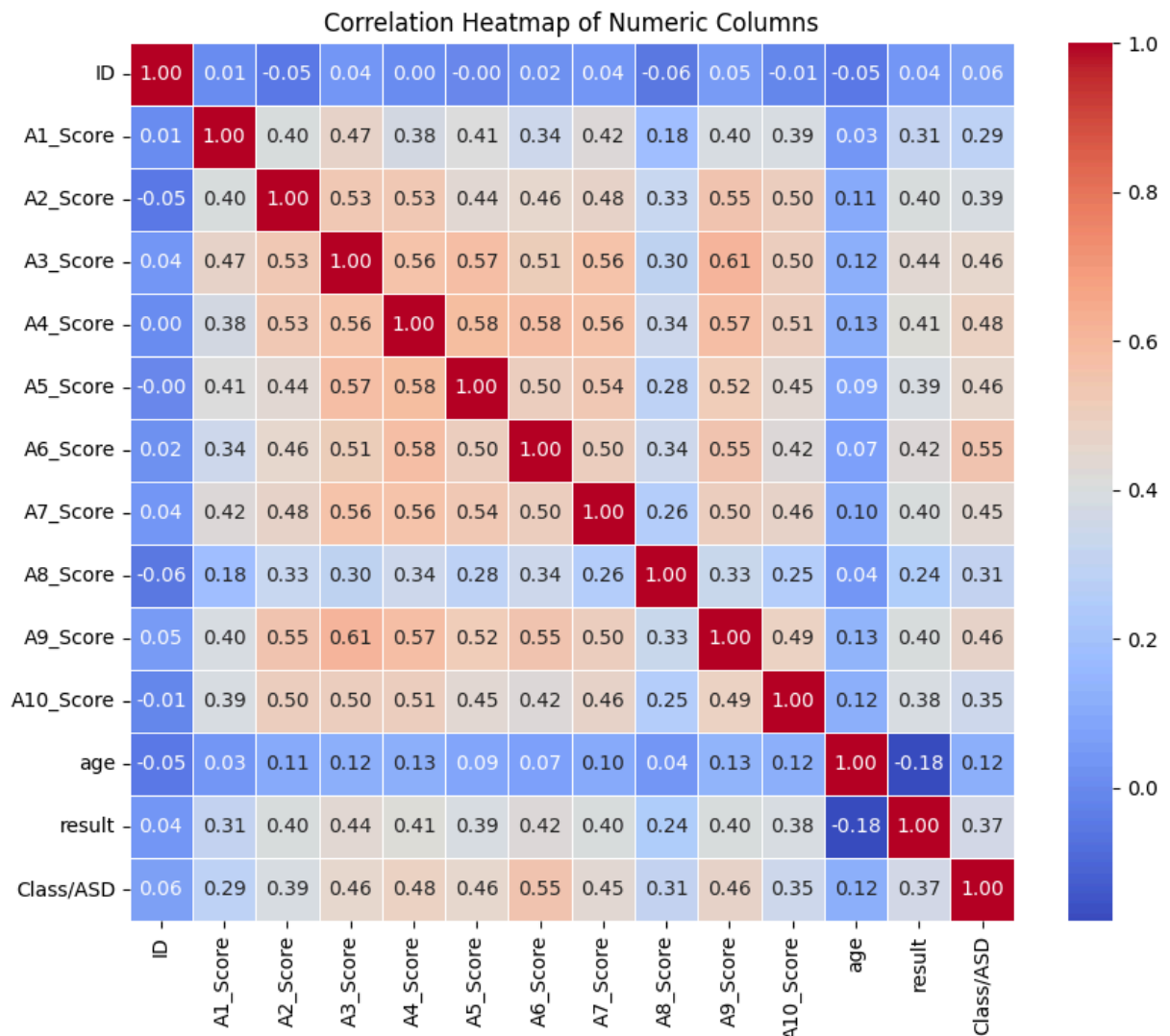
	colName	unique	dtype	categories
0	ID	597	int64	[3, 6, 7, 8, 9, 10, 14, 15, 17, 18, 20, 21, 22...
1	A1_Score	2	int64	[1, 0]
2	A2_Score	2	int64	[1, 0]
3	A3_Score	2	int64	[1, 0]
4	A4_Score	2	int64	[1, 0]
5	A5_Score	2	int64	[1, 0]
6	A6_Score	2	int64	[1, 0]
7	A7_Score	2	int64	[1, 0]
8	A8_Score	2	int64	[1, 0]
9	A9_Score	2	int64	[1, 0]
10	A10_Score	2	int64	[1, 0]
11	age	597	float64	[7.380373076, 31.5279644, 28.42797076, 26.4844...
12	gender	2	object	[Male, Female]
13	ethnicity	10	object	[White-European, Middle Eastern, Pasifika, Bla...
14	jaundice	2	object	[no, yes]
15	austim	2	object	[yes, no]
16	contry_of_res	55	object	[United States, Jordan, United Kingdom, Brazil...
17	used_app_before	2	object	[no, yes]
18	result	597	float64	[14.85148447, 9.562117428, 7.984569116, 13.237...
19	age_desc	1	object	[18 and more]
20	relation	6	object	[Self, Parent, Unknown, Relative, Others, Heal...
21	Class/ASD	2	int64	[1, 0]
22	gender_encoded	2	int32	[1, 0]
23	country_encoded	55	int32	[53, 30, 52, 12, 35, 51, 24, 31, 26, 20, 32, 5...
24	ethnicity_encoded	10	int32	[9, 4, 6, 1, 5, 2, 0, 8, 7, 3]
25	jaundice_encoded	2	int32	[0, 1]
26	austim_encoded	2	int32	[1, 0]

In []: `training_data.columns`

```
Out[ ]: Index(['ID', 'A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',  
             'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age',  
             'gender', 'ethnicity', 'jaundice', 'austim', 'contry_of_res',  
             'used_app_before', 'result', 'age_desc', 'relation', 'Class/ASD',  
             'gender_encoded', 'country_encoded', 'ethnicity_encoded',  
             'jaundice_encoded', 'austim_encoded'],  
          dtype='object')
```

EDA

```
In [ ]: # Select only numeric columns  
numeric_columns = training_data.select_dtypes(include=['int64', 'float64']).columns  
  
# Create a subset of DataFrame with numeric columns  
numeric_data = training_data[numeric_columns]  
  
# Calculate correlation matrix  
correlation_matrix = numeric_data.corr()  
  
# Plot heatmap  
plt.figure(figsize=(10, 8))  
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=  
plt.title('Correlation Heatmap of Numeric Columns')  
plt.show()
```



ANALYSIS:

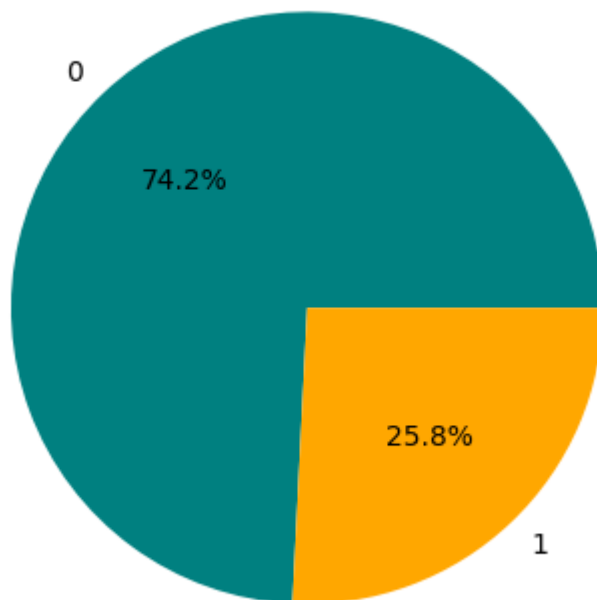
The following inferences can be drawn from the above heatmap:

1. There is a negative correlation between the columns 'age' and 'result', which suggests that as age increases, the possibility of a person being diagnosed with autism decreases. This means that this disease is detected at a very early age, possibly by 2 years.
2. There is a slightly negative correlation between 'gender_encoded' and 'A2_score', which suggests that the attribute score of a particular condition (A2_score) is found specifically in only one gender, which can be studied further in the analysis.
3. Again, there is a slightly negative correlation between 'gender' and 'country', which suggests that in some particular countries, specific genders are found to be diagnosed with autism.
4. A positive correlation can be seen between all the attribute scores 'A1, A2, A3..., A10' and 'age', which suggests that these attribute scores are dependent on the ages of the children.

5. Finally, 'Class/ASD' almost has a positive correlation with all the factors, which suggests that all the factors in the dataset are somewhat equally important in determining autism.

```
In [ ]: colors = ['#008080', '#FFA700', '#FF7F50']
# Calculate the value counts for 'Class/ASD'
class_counts = training_data['Class/ASD'].value_counts()

# Plot the pie chart
plt.pie(class_counts.values, labels=class_counts.index, autopct='%1.1f%%', colors=co
plt.show()
```

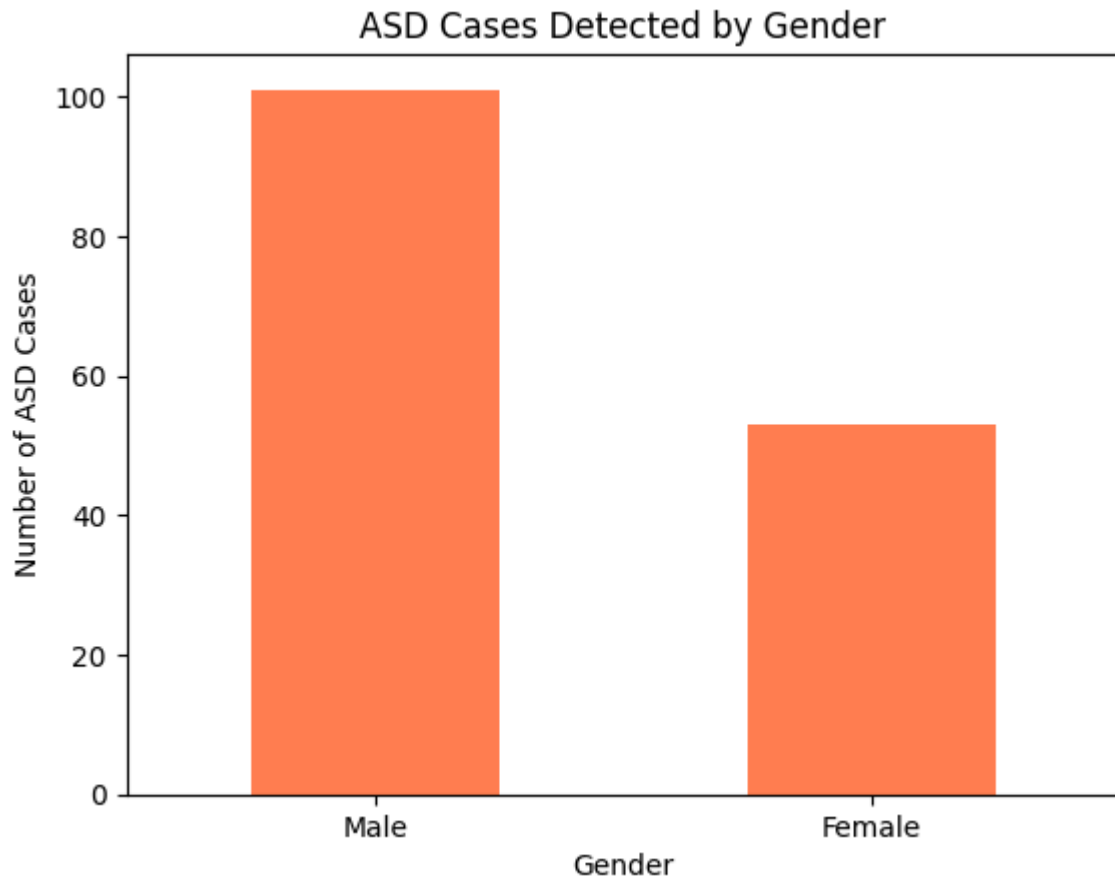


ANALYSIS:

The above piechart simply depicts the distribution of 1s and 0s, that is there are clearly more number of cases where no autism was detected and fewer cases where it was detected.

```
In [ ]: # Count the number of ASD cases for each gender
gender_asd_counts = training_data[training_data['Class/ASD'] == 1]['gender'].value_

# Plot the bar chart
gender_asd_counts.plot(kind='bar', color='coral')
plt.xlabel('Gender')
plt.ylabel('Number of ASD Cases')
plt.title('ASD Cases Detected by Gender')
plt.xticks(rotation=0) # Rotate x-axis labels if needed
plt.show()
```



ANALYSIS:

We can clearly see from the above bar graph that there have been more cases where ASD was detected in males than in females.

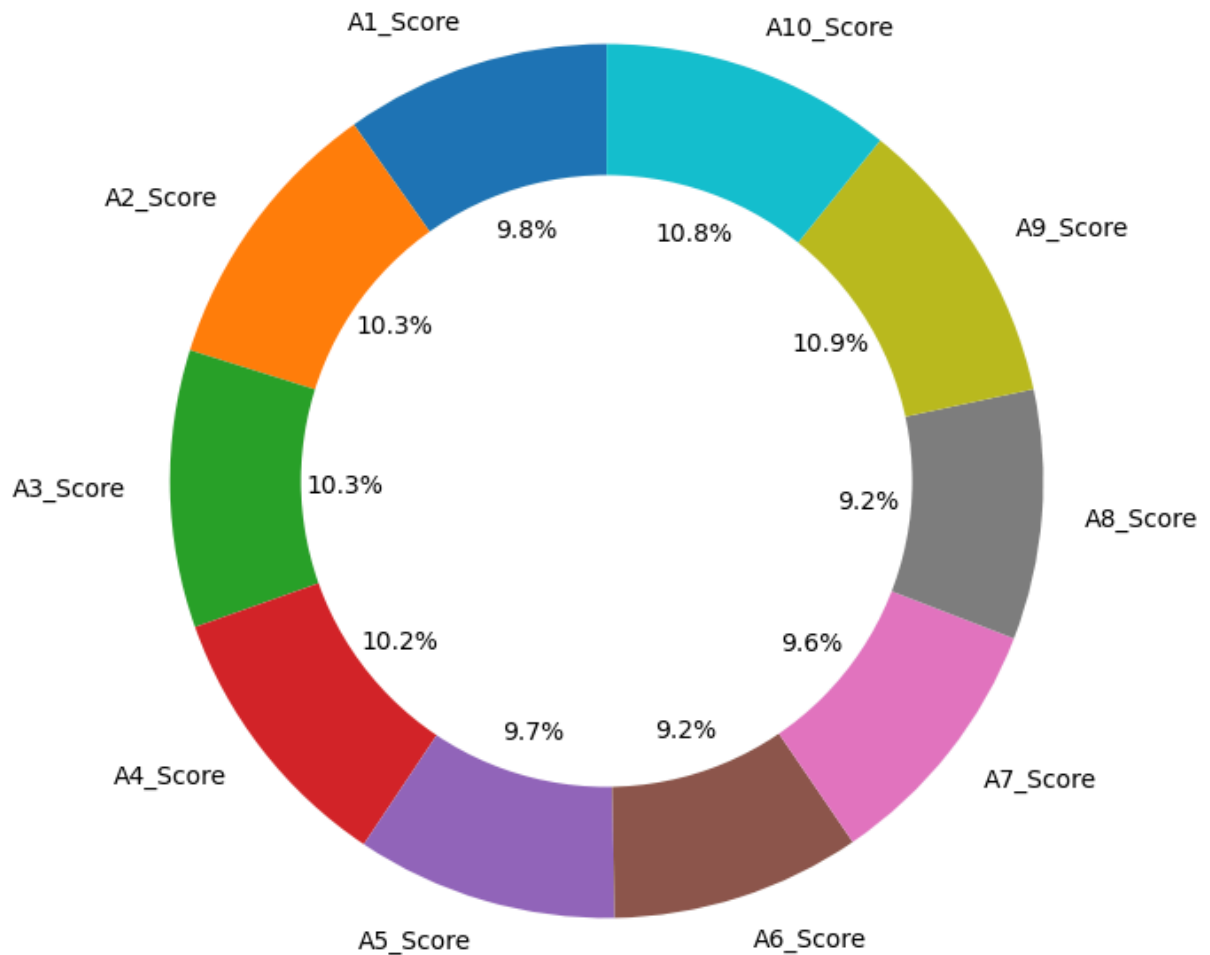
```
In [ ]: # Select ASD cases from the training data
asd_cases = training_data[training_data['Class/ASD'] == 1]

# Select only the columns containing A1 to A10 scores
a_scores = asd_cases[['A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
                      'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score']]

# Calculate the sum of scores for each attribute
attribute_totals = a_scores.sum()

# Plot the doughnut chart
plt.figure(figsize=(8, 8))
plt.pie(attribute_totals, labels=attribute_totals.index, autopct='%1.1f%%', startangle=90)
centre_circle = plt.Circle((0,0),0.70,fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title('Distribution of A1 to A10 Scores for ASD Cases')
plt.show()
```


Distribution of A1 to A10 Scores for ASD Cases

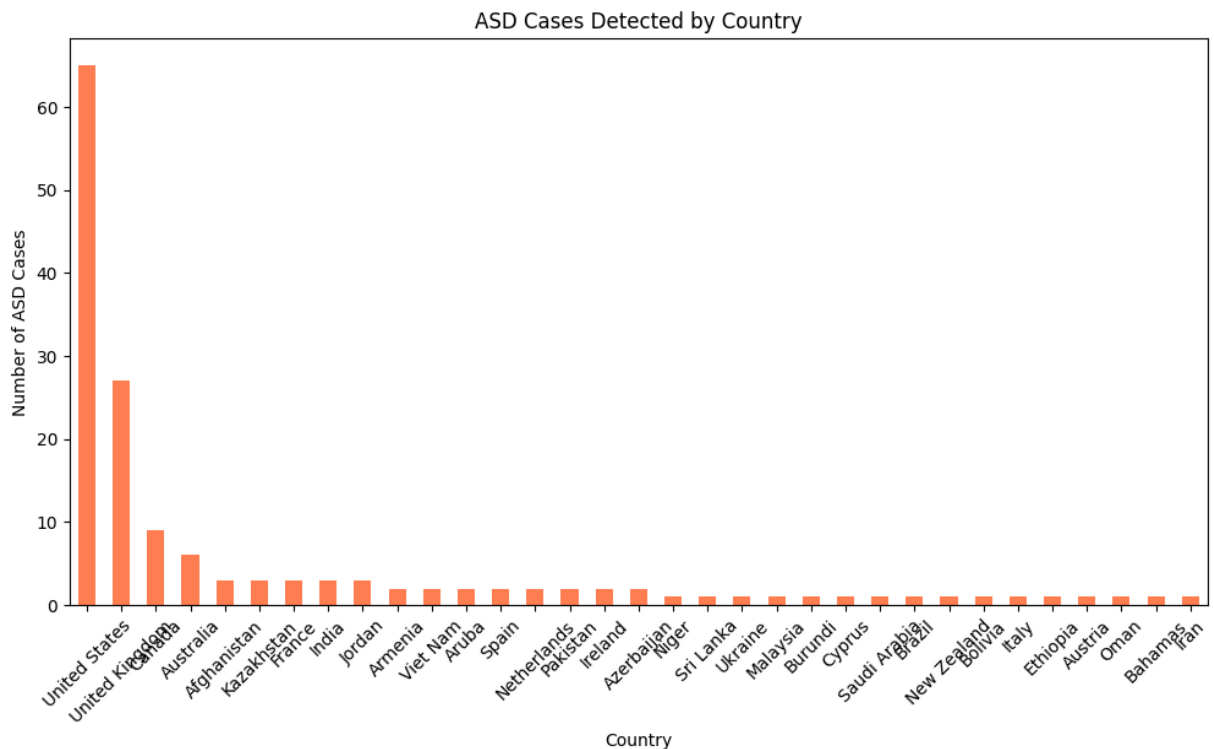


ANALYSIS:

From the above doughnut graph, we can decipher that not all the attribute scores have same kind of impact on te ASD detection. As per the relation, we can see that A9 has the maximum correlation, followed by A10,A2,A3 and so on and A6 and A7 have the least correlation amongst all.

```
In [ ]: # Count the number of ASD cases for each country
country_asd_counts = training_data[training_data['Class/ASD'] == 1]['contry_of_res']

# Plot the bar chart
plt.figure(figsize=(12, 6))
country_asd_counts.plot(kind='bar', color='coral')
plt.xlabel('Country')
plt.ylabel('Number of ASD Cases')
plt.title('ASD Cases Detected by Country')
plt.xticks(rotation=45)
plt.show()
```



The above bar graph represents the distribution of ASD Cases country-wise. it can be hence deciphered that the United states have the maximum number of cases, followed by the United Kingdom, Canada, Australia and Afghanistan. Whereas the least amount of cases have been recorded in Iran, Bahamas, Oman, Austria and Ethiopia.

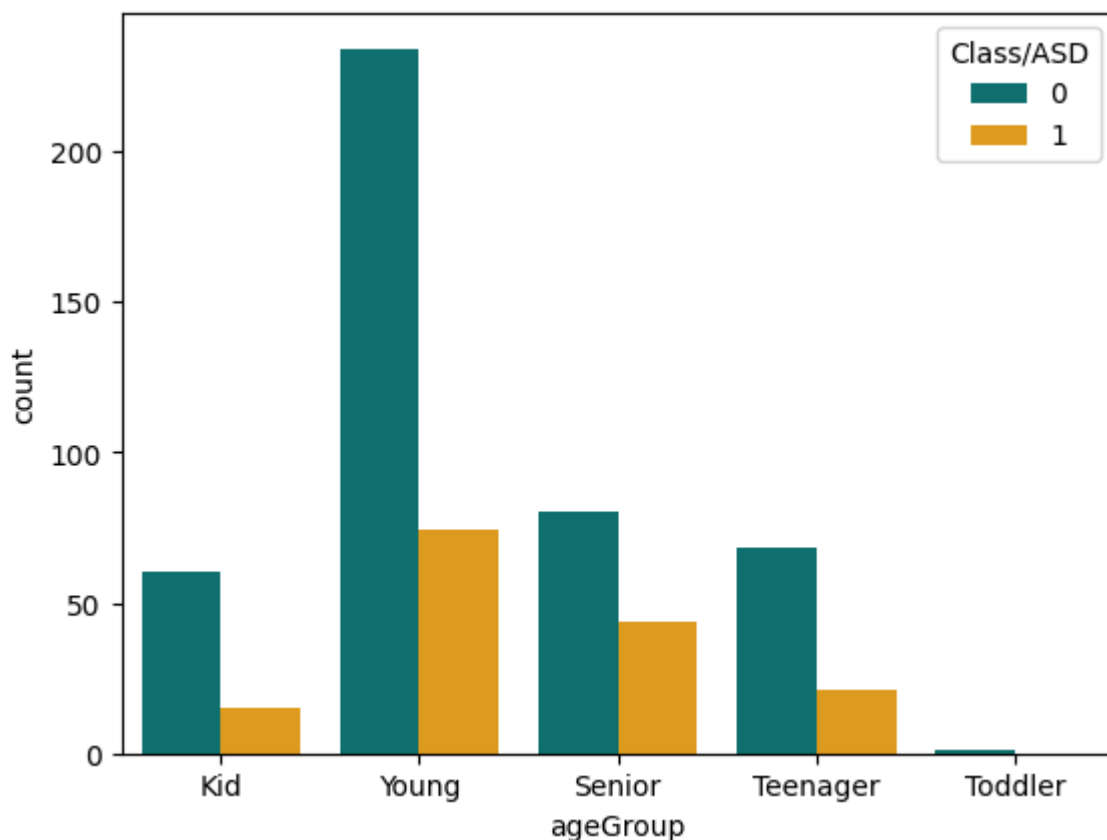
```
In [ ]: def convertAge(age):
        if age < 4:
            return 'Toddler'
        elif age < 12:
            return 'Kid'
        elif age < 18:
            return 'Teenager'
        elif age < 40:
            return 'Young'
        else:
            return 'Senior'

        training_data['ageGroup'] = training_data['age'].apply(convertAge)
```

```
In [ ]: colors = ['#008080', '#FFA700', '#FF7F50']

sns.countplot(x=training_data['ageGroup'], hue=training_data['Class/ASD'], palette=
plt.show())
```

C:\Users\Neevisha\AppData\Local\Temp\ipykernel_19308\1400967493.py:3: UserWarning: The palette list has more values (3) than needed (2), which may not be intended.
 sns.countplot(x=training_data['ageGroup'], hue=training_data['Class/ASD'], palette=colors)



ANALYSIS:

The above coutplot shows a distribution od ASD across different age groups. People with ages between 18 and 40 that is the young people have shown more number of cases of this disorder, followed by the Senior age group, that is people having ages more than 40 years. It can also be seen that toddlers haverecorded the least number of cases followed by kids and then teenagers.

```
In [ ]: training_data.columns
```

```
Out[ ]: Index(['ID', 'A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
              'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age',
              'gender', 'ethnicity', 'jaundice', 'austim', 'contry_of_res',
              'used_app_before', 'result', 'age_desc', 'relation', 'Class/ASD',
              'gender_encoded', 'country_encoded', 'ethnicity_encoded',
              'jaundice_encoded', 'austim_encoded', 'ageGroup'],
              dtype='object')
```

```
In [ ]: # Set up the figure with subplots
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 6))

# Plot violin plot for ethnicity_encoded vs. ASD diagnosis
sns.violinplot(x='Class/ASD', y='ethnicity_encoded', data=training_data, ax=axes[0])
axes[0].set_xlabel('ASD Diagnosis')
axes[0].set_ylabel('Ethnicity (Encoded)')
axes[0].set_title('Ethnicity (Encoded) vs. ASD Diagnosis')

# Plot violin plot for jaundice_encoded vs. ASD diagnosis
```

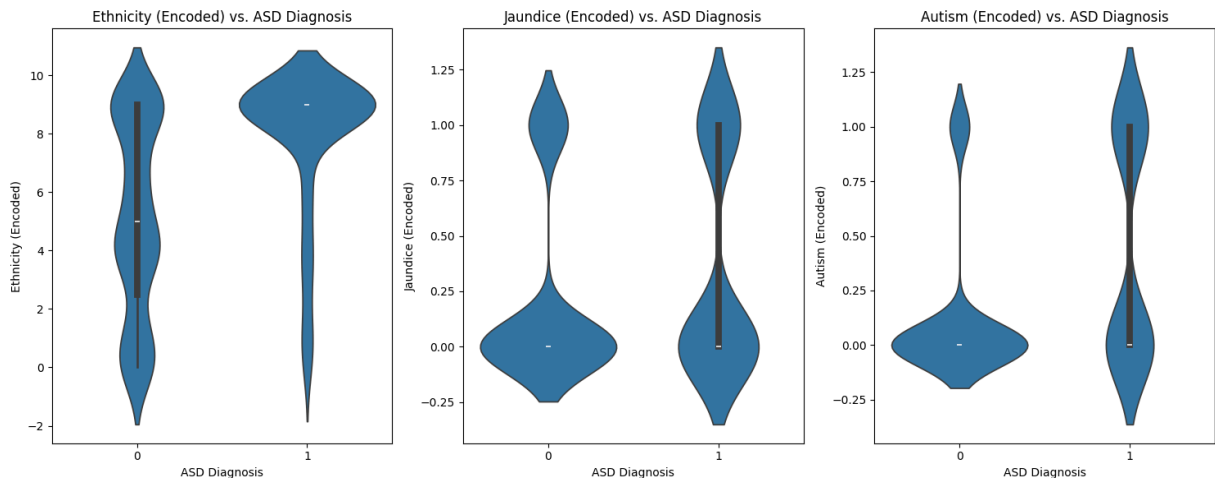
```

sns.violinplot(x='Class/ASD', y='jaundice_encoded', data=training_data, ax=axes[1])
axes[1].set_xlabel('ASD Diagnosis')
axes[1].set_ylabel('Jaundice (Encoded)')
axes[1].set_title('Jaundice (Encoded) vs. ASD Diagnosis')

# Plot violin plot for autism_encoded vs. ASD diagnosis
sns.violinplot(x='Class/ASD', y='austim_encoded', data=training_data, ax=axes[2])
axes[2].set_xlabel('ASD Diagnosis')
axes[2].set_ylabel('Autism (Encoded)')
axes[2].set_title('Autism (Encoded) vs. ASD Diagnosis')

plt.tight_layout()
plt.show()

```



ANALYSIS:

The above is a violin plot which shows the correlation of columns like history of jaundice(jaundice_encoded), history of autism(austim_encoded) and ethnicity(ethnicity_encoded) with the likelihood of being detected with ASD. The areas where there is a fatter part shows that there is more density and similarly the thinner part depicts lower density.

- **Ethnicity:** There appears to be a slightly higher density of ASD diagnoses for individuals with encoded ethnicity values of 2, 4, and 6 when compared to other values. However, the differences are not substantial, suggesting that ethnicity may not be a strong predictor of ASD.
- **Jaundice:** Individuals with encoded jaundice values of 0 and 1 seem to have a higher likelihood of being diagnosed with ASD compared to those with values of 2 and 3. This could indicate a potential association between jaundice and ASD, although further investigation is needed.
- **Autism:** The violin plot for autism shows a clear distinction between the encoded values. Individuals with an encoded autism value of 1 have a significantly higher density of ASD diagnoses compared to those with values of 0 and 2. This suggests a strong

association between a history of autism and the likelihood of being diagnosed with ASD.

FITTING THE MODEL

```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score

        from sklearn.linear_model import LogisticRegression
        from sklearn.svm import SVC
        from sklearn.ensemble import GradientBoostingClassifier
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.tree import DecisionTreeClassifier
```

```
In [ ]: columns_to_drop = ['ID', 'gender', 'contry_of_res', 'ethnicity',
                          'jaundice', 'austim', 'ageGroup', 'used_app_before', 'age_desc', 'relation', 'Cla
```

```
In [ ]: X = training_data.drop(columns=columns_to_drop)
        y = training_data['Class/ASD']
```

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
```

```
In [ ]: # Dictionary to hold model accuracies
        accuracies = {}
```

Fitting the model using SVC- Support Vector Classifier

```
In [ ]: svc_model = SVC()
        svc_model.fit(X_train, y_train)
        svc_pred = svc_model.predict(X_test)
        svc_accuracy = accuracy_score(y_test, svc_pred)
        accuracies['SVC'] = svc_accuracy
```

Fitting the model using Logistic Regression

```
In [ ]: lr_model = LogisticRegression(max_iter=1000)
        lr_model.fit(X_train, y_train)
        lr_pred = lr_model.predict(X_test)
        lr_accuracy = accuracy_score(y_test, lr_pred)
        accuracies['Logistic Regression'] = lr_accuracy
```

Fitting the model using Random Forest

```
In [ ]: rf_model = RandomForestClassifier()
        rf_model.fit(X_train, y_train)
        rf_pred = rf_model.predict(X_test)
        rf_accuracy = accuracy_score(y_test, rf_pred)
        accuracies['Random Forest'] = rf_accuracy
```

Fitting the model using XGBoost

```
In [ ]: xgb_model = GradientBoostingClassifier()
xgb_model.fit(X_train, y_train)
xgb_pred = xgb_model.predict(X_test)
xgb_accuracy = accuracy_score(y_test, xgb_pred)
accuracies['XGBoost'] = xgb_accuracy
```

Fitting the model using Decision Tree

```
In [ ]: dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
accuracies['DecisionTree'] = dt_accuracy
```

Getting the accuracies for all the models

```
In [ ]: for model, accuracy in accuracies.items():
        print(f"{model} accuracy: {accuracy:.2f}")
```

```
SVC accuracy: 0.68
Logistic Regression accuracy: 0.76
Random Forest accuracy: 0.77
XGBoost accuracy: 0.74
DecisionTree accuracy: 0.72
```

After fitting the above models, we know that SVC is not a suitable model to use as its accuracy is only 68%.

Rest all the other models, i.e Logistic Regression or XGBoost or Random Forest can be used.

However, we will continue with Random Forest as it shows the highest accuracy of 78%.

Confusion matrix for all the models to check which performs the best

```
In [ ]: from sklearn.metrics import classification_report, confusion_matrix

# Logistic Regression
lr_report = classification_report(y_test, lr_pred, zero_division=1) # Set zero_div
lr_confusion_matrix = confusion_matrix(y_test, lr_pred)

print("Logistic Regression Evaluation Metrics:")
print(lr_report)
print("Confusion Matrix:")
print(lr_confusion_matrix)

# Support Vector Classifier (SVC)
svc_report = classification_report(y_test, svc_pred, zero_division=1) # Set zero_d
svc_confusion_matrix = confusion_matrix(y_test, svc_pred)

print("\nSupport Vector Classifier (SVC) Evaluation Metrics:")
print(svc_report)
print("Confusion Matrix:")
print(svc_confusion_matrix)
```

```
# Random Forest
rf_report = classification_report(y_test, rf_pred, zero_division=1) # Set zero_div
rf_confusion_matrix = confusion_matrix(y_test, rf_pred)

print("\nRandom Forest Evaluation Metrics:")
print(rf_report)
print("Confusion Matrix:")
print(rf_confusion_matrix)

# Decision Tree
dt_report = classification_report(y_test, dt_pred, zero_division=1) # Set zero_div
dt_confusion_matrix = confusion_matrix(y_test, dt_pred)

print("\nDecision Tree Evaluation Metrics:")
print(dt_report)
print("Confusion Matrix:")
print(dt_confusion_matrix)

# XGBoost
xgb_report = classification_report(y_test, xgb_pred, zero_division=1) # Set zero_d
xgb_confusion_matrix = confusion_matrix(y_test, xgb_pred)

print("\nXGBoost Evaluation Metrics:")
print(xgb_report)
print("Confusion Matrix:")
print(xgb_confusion_matrix)
```

Logistic Regression Evaluation Metrics:

	precision	recall	f1-score	support
0	0.85	0.78	0.82	82
1	0.60	0.71	0.65	38
accuracy			0.76	120
macro avg	0.73	0.75	0.73	120
weighted avg	0.77	0.76	0.76	120

Confusion Matrix:

```
[[64 18]
 [11 27]]
```

Support Vector Classifier (SVC) Evaluation Metrics:

	precision	recall	f1-score	support
0	0.72	0.89	0.79	82
1	0.50	0.24	0.32	38
accuracy			0.68	120
macro avg	0.61	0.56	0.56	120
weighted avg	0.65	0.68	0.64	120

Confusion Matrix:

```
[[73  9]
 [29  9]]
```

Random Forest Evaluation Metrics:

	precision	recall	f1-score	support
0	0.82	0.84	0.83	82
1	0.64	0.61	0.62	38
accuracy			0.77	120
macro avg	0.73	0.72	0.73	120
weighted avg	0.76	0.77	0.76	120

Confusion Matrix:

```
[[69 13]
 [15 23]]
```

Decision Tree Evaluation Metrics:

	precision	recall	f1-score	support
0	0.79	0.79	0.79	82
1	0.55	0.55	0.55	38
accuracy			0.72	120
macro avg	0.67	0.67	0.67	120
weighted avg	0.72	0.72	0.72	120

Confusion Matrix:

```
[[65 17]
 [17 21]]
```


XGBoost Evaluation Metrics:

	precision	recall	f1-score	support
0	0.82	0.79	0.81	82
1	0.59	0.63	0.61	38
accuracy			0.74	120
macro avg	0.70	0.71	0.71	120
weighted avg	0.75	0.74	0.74	120

Confusion Matrix:

```
[[65 17]
 [14 24]]
```

The following code performs hyperparameter tuning for a Random Forest classifier using grid search with cross-validation and then evaluates the best model on the testing data.

```
In [ ]: import warnings
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Suppress all warnings
warnings.filterwarnings("ignore")

# Generate synthetic data for demonstration
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_class

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Define the hyperparameters to tune
param_grid = {
    'n_estimators': [100, 200],      # Decreasing number of trees
    'max_depth': [None, 5, 10],      # Adjusting maximum depth of the trees
    'min_samples_split': [2, 3],     # Adjusting minimum samples required to split
    'min_samples_leaf': [1, 2],      # Adjusting minimum samples required at each
    'max_features': ['auto', 'sqrt'] # Number of features to consider when looking

}

# Initialize Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42)

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=rf_classifier, param_grid=param_grid, cv=5, scoring=
grid_search.fit(X_train, y_train)

# Evaluate the best model on the testing data
best_model = grid_search.best_estimator_
training_accuracy = best_model.score(X_train, y_train)
print("Accuracy on Training Data:", training_accuracy)

# Evaluate the best model on the testing data
```

```
testing_accuracy = best_model.score(X_test, y_test)
print("Accuracy on Testing Data:", testing_accuracy)
```

Accuracy on Training Data: 1.0

Accuracy on Testing Data: 0.92

Analysis of the Performance: A testing accuracy of 92% suggests that the model performs well on unseen data. It correctly predicts the class labels for approximately 92% of the samples in the testing dataset.

Model Effectiveness: The accuracy of 92% indicates that the model is effective at identifying patterns and relationships in the data. It demonstrates that the model has learned from the training data and can make nearly accurate predictions on new, unseen examples.

```
In [ ]: from sklearn.feature_selection import SelectFromModel
```

```
In [ ]: import numpy as np
        from sklearn.datasets import make_classification
        from sklearn.model_selection import train_test_split
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.decomposition import PCA
        from sklearn.metrics import accuracy_score
        from sklearn.feature_selection import SelectFromModel
        import pandas as pd

        # Generate synthetic data with 17 features
        X, y = make_classification(n_samples=1000, n_features=17, n_classes=2, random_state=42)

        # Train a Random Forest classifier
        rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
        rf_classifier.fit(X, y)

        # Calculate feature importance
        feature_importances = rf_classifier.feature_importances_

        # Assuming 'feature_names' is defined correctly
        feature_names = ['A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score', 'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age', 'result', 'gender_encoded', 'country_encoded', 'ethnicity_encoded', 'jaundice_encoded', 'austim_encoded']

        # Create a DataFrame to store feature importance
        feature_importances_df = pd.DataFrame({'feature_name': feature_names, 'importance': feature_importances})

        # Sort features by importance
        feature_importances_df = feature_importances_df.sort_values(by='importance', ascending=False)

        # Select top 5 features
        top_features = feature_importances_df['feature_name'].head(5).tolist()

        print("Top 5 Features:")
        for feature in top_features:
            print(feature)
```

```

# Fit PCA to the training data
pca = PCA(n_components=5)
pca.fit(X)

# Get the top features contributing to each principal component
n_top_features = 3 # Number of top features to print for each component
for i, component in enumerate(pca.components_):
    top_feature_indices = np.argsort(np.abs(component))[-n_top_features:]
    top_feature_names = [feature_names[index] for index in top_feature_indices]
    print(f"Top features for Component {i+1}: {'', ' '.join(top_feature_names)}")

# Transform the data using selected features
X_selected = SelectFromModel(rf_classifier, threshold='median').fit_transform(X, y)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size=0.2, r

# Fit a Random Forest classifier to the PCA-transformed data
rf_pca = RandomForestClassifier(random_state=42)
rf_pca.fit(X_train, y_train)
y_pred_pca = rf_pca.predict(X_test)
accuracy_pca = accuracy_score(y_test, y_pred_pca)
print("PCA Accuracy:", accuracy_pca)

```

Top 5 Features:

A2_Score

austim_encoded

A1_Score

A3_Score

age

Top features for Component 1: austim_encoded, A3_Score, A2_Score

Top features for Component 2: A3_Score, A2_Score, A1_Score

Top features for Component 3: A5_Score, A9_Score, A7_Score

Top features for Component 4: country_encoded, jaundice_encoded, gender_encoded

Top features for Component 5: A8_Score, A6_Score, result

PCA Accuracy: 0.895

```

In [ ]: import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score
from sklearn.feature_selection import SelectFromModel
import pandas as pd

# Generate synthetic data with 17 features
X, y = make_classification(n_samples=1000, n_features=17, n_classes=2, random_state

# Train a Random Forest classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X, y)

# Calculate feature importance
feature_importances = rf_classifier.feature_importances_

```

```

# Assuming 'feature_names' is defined correctly
feature_names = ['A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score', 'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age', 'result', 'gender_encoded', 'country_encoded', 'ethnicity_encoded', 'jaundice_encoded', 'austim_encoded']

# Create a DataFrame to store feature importance
feature_importances_df = pd.DataFrame({'feature_name': feature_names, 'importance': 

# Sort features by importance
feature_importances_df = feature_importances_df.sort_values(by='importance', ascend

# Select top 5 features
top_features = feature_importances_df['feature_name'].head(5).tolist()

print("Top 5 Features:")
for feature in top_features:
    print(feature)

# Fit PCA to the training data
pca = PCA(n_components=5)
pca.fit(X)

# Get the top features contributing to each principal component
n_top_features = 3 # Number of top features to print for each component
for i, component in enumerate(pca.components_):
    top_feature_indices = np.argsort(np.abs(component))[-n_top_features:]
    top_feature_names = [feature_names[index] for index in top_feature_indices]
    print(f"Top features for Component {i+1}: {' '.join(top_feature_names)}")

# Transform the data using selected features
X_selected = SelectFromModel(rf_classifier, threshold='median').fit_transform(X, y)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size=0.2, r

# Fit a Random Forest classifier to the PCA-transformed data
rf_pca = RandomForestClassifier(random_state=42)
rf_pca.fit(X_train, y_train)
y_pred_pca = rf_pca.predict(X_test)
accuracy_pca = accuracy_score(y_test, y_pred_pca)
print("PCA Accuracy:", accuracy_pca)

```

Top 5 Features:

A2_Score

austim_encoded

A1_Score

A3_Score

age

Top features for Component 1: austim_encoded, A3_Score, A2_Score

Top features for Component 2: A3_Score, A2_Score, A1_Score

Top features for Component 3: A5_Score, A9_Score, A7_Score

Top features for Component 4: country_encoded, jaundice_encoded, gender_encoded

Top features for Component 5: A8_Score, A6_Score, result

PCA Accuracy: 0.895

Deploying Model

```
In [ ]: import pickle
```

```
In [ ]: pickle.dump(rf_classifier, open('rf_classifier_selected.pkl', 'wb'))
```

```
In [ ]: # Load the model  
with open('rf_classifier_selected.pkl', 'rb') as file:  
    model = pickle.load(file)
```